

MedVerify AI — Detailed Implementation Plan

Datasets, Models, Architecture, and Phased Delivery Roadmap

Built on the MedVerify AI reference design, corrected and extended against the gaps identified in the Research Gap Discovery & Novel Research Proposal review of 4 IEEE papers.

How This Document Is Built

This plan takes the uploaded MedVerify AI reference design as its starting architecture, critically evaluates it (Section 1), then produces a corrected, de-scoped, and evidence-grounded implementation plan. Every design decision below is traced either to a strength worth keeping from the reference document or to a gap identified in the earlier four-paper research-gap review — both are cited inline as [MedVerify Ref] or [Gap #] respectively.

1. Critical Analysis of the MedVerify AI Reference Document

The reference document is a strong architectural concept — stronger than any single paper in the prior four-paper corpus, because it already proposes an evidence-reliability ranking system and a consensus engine, which directly answers Gap 4.1 (no source-reliability weighting) from the earlier review. However, as a document it is a wish-list of 12 modules and 12+ disease categories with no phasing, no evaluation protocol, and no mechanism to prevent its own LLM layer from hallucinating — the same blind spot found in Papers 2 and 4 of the earlier review. The table below separates what to keep as-is, what to fix, and what to add.

1.1 What to Keep (genuine strengths)

Reference Feature	Why It's Worth Keeping
Evidence Ranking Engine (WHO 98, Systematic Review 95, RCT 90 ... Blog 0)	Directly implements reliability-weighted evidence, the single highest-leverage gap identified across all four prior papers (none of them ranked evidence by source quality).
Medical Consensus Engine (supporting/contradicting/neutral study counts)	More defensible than a single binary verdict; mirrors how real systematic reviews are read.
Hybrid RAG (static trusted corpus + live PubMed API)	Solves the “static knowledge after training” problem named in every prior paper's future-work section.
Open-weight LLM choice (Qwen 3 / Llama 3.1 / Gemma 3) instead of GPT-4-only	Avoids the vendor lock-in and per-call cost risk flagged as a practical gap in Papers 2 and 4 of the prior review.
Named, real benchmark datasets (PubHealth, SciFact, CoAID, HealthFact, MedNLI)	These are legitimate, citable academic datasets — a major upgrade over Papers 2 and 3, which used no dataset at all.

1.2 What to Fix Before Building

Issue in Reference Document	Why It's a Problem	Fix Applied in This Plan
Evidence-quality scores (98, 95, 90...) are asserted, not derived	No stated methodology for how these numbers were chosen; unvalidated weights can silently bias every verdict	Section 6 defines the weighting as a configurable, versioned scoring function validated by a calibration study (Section 9)
12 disease categories + multimodal + multilingual + consensus engine	Scope far exceeds what a single team can build and evaluate credibly — the same over-	Section 3 defines a 3-phase MVP: 3 diseases, text-only, single language for Phase 1

Issue in Reference Document	Why It's a Problem	Fix Applied in This Plan
+ dashboard + admin panel, all in one prototype	promising pattern seen in Paper 4 (claims multimodal, never implements it)	
No hallucination/faithfulness check on LLM-generated explanations	The exact blind spot identified in Papers 2 and 4 of the prior review — an LLM can generate a fluent explanation that isn't actually supported by the retrieved evidence	Section 7 adds a mandatory Faithfulness Verification module between the LLM and the dashboard
No evaluation plan for the system as a whole (datasets are named but not mapped to train/val/test use)	Without this, credibility/consensus scores cannot be validated before shipping	Section 9 defines dataset-to-metric mapping and an evaluation protocol with baselines
Kubernetes proposed for a prototype	Operational overhead disproportionate to an MVP's actual load	Section 10 recommends Docker Compose for Phase 1–2, Kubernetes deferred to Phase 3 scale-up
No adversarial, privacy, or data-poisoning considerations	Same security gap found across all four prior papers; a Weaviate/FAISS-style fact store with no integrity checks is a single point of failure	Section 11 adds a security and privacy plan, including vector-store integrity monitoring

2. Problem Statement, Existing Solutions, Proposed Solution (Refined)

2.1 Problem Statement

Health-related misinformation spreads faster than it can be manually fact-checked, and current AI verification approaches are fragmented: supervised classifiers lack evidence grounding, LLM-prompting systems lack rigorous evaluation, and hybrid retrieval systems — including the MedVerify AI reference design in its current form — do not verify that their own generated explanations are actually supported by the evidence they retrieved. This leaves users unable to distinguish a well-reasoned, evidence-backed verdict from a fluent but unsupported LLM statement.

2.2 Existing Solutions and Their Specific Failure Modes

- Supervised classifiers (BERT/DistilBERT-family): accurate on narrow, static, single-disease datasets, but produce a black-box True/False label with no evidence, no references, and no way to update as research evolves.
- Pure LLM-prompting systems (GPT-4 zero/few-shot): dataset-free and multilingual-capable, but evaluated with weak or task-mismatched metrics and no check on whether the model's reasoning is grounded in real evidence.
- Hybrid retrieval + LLM systems (including the MedVerify AI reference design as written): the most complete approach on paper, combining live retrieval, evidence ranking, and consensus analysis — but still missing a verification step confirming the final explanation text is faithful to the retrieved evidence, and currently scoped far beyond what can be reliably built and evaluated in a first release.

2.3 Proposed Solution

A phased implementation of MedVerify AI that keeps the reference design's strongest ideas — reliability-weighted evidence ranking, hybrid static+live retrieval, and consensus analysis — while adding a Faithfulness Verification layer that checks every LLM-generated explanation sentence against the retrieved evidence before it reaches the user, and de-scoping the first release to a small, provably correct MVP (3 disease categories, text-only claims, single language) before expanding to the full 12-module vision.

3. MVP Scope Definition (Phase 1)

The reference document's full scope (12 disease categories, text+image+video input, multilingual, full dashboard + admin panel, continuous auto-updating knowledge base) is a strong Phase 3 vision, not a buildable MVP. The table below defines what ships in Phase 1.

Dimension	Reference Document Scope	Phase 1 MVP Scope (this plan)
Disease categories	10–12 (Diabetes, Cancer, Cardiovascular, Vaccination, Infectious Disease, Nutrition, Mental Health, Kidney, Liver, Women's Health)	3: Diabetes, Cardiovascular Disease, Vaccination — chosen because they have the strongest coverage in the recommended benchmark datasets (Section 5)
Input modality	Text + Image (OCR) + Video (Speech-to-Text)	Text only; OCR and Speech-to-Text moved to Phase 2 (architecture designed to accept them later without rework, Section 6)
Language	Multilingual	English only for Phase 1; Arabic added in Phase 2 given the demonstrated need from the prior research review
Evidence sources	WHO, CDC, ICMR, NHS, PubMed, PubMed Central (static + live)	PubMed + PubMed Central (live API) + a small curated WHO/CDC static guideline set for the 3 chosen diseases
Retrieval store	Vector database (unspecified in general text) + FAISS in tech stack	FAISS for Phase 1 (simplest, no server to run); migration path to Qdrant/Weaviate noted for Phase 3 when metadata-filtering and access control are needed
Explainability	LLM explanation + references	LLM explanation + references + mandatory Faithfulness Verification pass (new, Section 7)
Deployment	Docker + Kubernetes	Docker Compose (single-host) for Phase 1–2; Kubernetes only if Phase 3 scale requires it

4. Phased Implementation Roadmap

PHASE 1 (Weeks 1–8) — Text-Only MVP, 3 Diseases, English

- Weeks 1–2: Data acquisition — download and preprocess PubHealth, SciFact, CoAID, HealthFact, MedNLI; scrape/curate WHO & CDC guideline PDFs for the 3 chosen diseases.
- Weeks 2–3: Build the static knowledge base — chunk documents, generate embeddings (BGE-large), load into FAISS.
- Weeks 3–4: Claim extraction + disease classification pipeline (fine-tune BioBERT for disease classification on the curated set).
- Weeks 4–5: Hybrid retrieval — combine FAISS similarity search with live PubMed E-utilities API calls.
- Weeks 5–6: Evidence Ranking Engine (calibrated scoring function, Section 6.3) + Consensus Engine (support/contradict/neutral tally).
- Weeks 6–7: LLM explanation generation (Llama 3.1 8B or Qwen 3 8B, locally hosted) + Faithfulness Verification module (Section 7).
- Weeks 7–8: Minimal web dashboard (claim input → verdict → evidence → explanation → references) + evaluation run against held-out test splits (Section 9).

PHASE 2 (Weeks 9–16) — Multimodal + Multilingual + Expanded Diseases

- Add OCR ingestion (Tesseract or a VLM-based OCR) and Speech-to-Text ingestion (Whisper) feeding into the same claim-extraction pipeline.
- Add Arabic-language support (dataset collection + evaluation, following the dialectal-transfer approach recommended in the prior research review).
- Expand disease coverage from 3 to 8–10 categories, re-running the calibration study (Section 9) for each newly added category before enabling it in production.
- Add user accounts, verification history, and feedback collection (PostgreSQL schema, Section 8).

PHASE 3 (Weeks 17–24+) — Scale, Automation, and Trust Infrastructure

- Automatic knowledge updating scheduler (periodic PubMed sync, embedding regeneration, vector-store versioning).
- Migrate vector store from FAISS to a metadata-filtering-capable store (Qdrant/Weaviate) to support the growing evidence-type/date/disease filters needed at scale.
- Add vector-store integrity monitoring (poisoning/anomaly detection, Section 11) before allowing automatic knowledge updates to run unsupervised.
- Move deployment from Docker Compose to Kubernetes only if load testing shows single-host limits are actually being reached.
- Build the analytics/misinformation-trend dashboard and admin panel.

5. Datasets — Detailed Plan

The reference document correctly names five real, citable benchmark datasets but does not specify how each is used. The table below assigns each dataset a concrete role (fine-tuning, evaluation, or knowledge-base seeding) mapped to a specific pipeline component.

Dataset	Description	Size (approx.)	Role in This Plan
PubHealth	Public-health claims paired with veracity labels (true/false/mixture/unproven) and journalist-written explanations	~11,800 claims	Primary fine-tuning + evaluation set for the verdict-classification component and for training/testing the Faithfulness Verification module (explanation-to-evidence alignment)
SciFact	Scientific claims paired with abstracts labeled as supporting/refuting/no-evidence, with rationale sentences	~1,400 claims, ~5,000 abstracts	Evaluation set for the Evidence Ranking and Consensus Engine, since it provides sentence-level supporting/refuting labels ideal for calibrating rank scores
CoAID	COVID-19 misinformation dataset covering news, claims, and social-media posts with true/false labels	~5,000 claims, ~296,000 social posts	Domain-adjacent evaluation set for the Vaccination/Infectious-Disease category and for stress-testing the claim-extraction module on noisy social-media-style text
HealthFact (PUBHEALTH variant used in some literature)	Health-claim fact-checking dataset with evidence documents and veracity labels	~11,000 claims	Secondary fine-tuning set to increase disease-category diversity beyond PubHealth's coverage
MedNLI	Clinical natural language inference dataset (entailment/contradiction/neutral) derived from clinical notes	~14,000 sentence pairs	Used specifically to train/validate the Faithfulness Verification module's entailment check (does the explanation sentence follow from the evidence sentence?), since this is exactly an NLI task

5.1 Knowledge-Base (Retrieval Corpus) Sources

Source	Content Type	Access Method	Update Cadence in This Plan
PubMed / PubMed Central	Peer-reviewed abstracts and open-access full papers	NCBI E-utilities API (live) + periodic bulk baseline download for initial indexing	Live query at verification time (Phase 1); scheduled incremental sync (Phase 3)
WHO fact sheets & guidelines	Official disease/vaccine guidance documents	Manually curated PDF/HTML set for the Phase 1 disease categories, expanded per category in Phase 2	Manual quarterly review (Phase 1–2); scheduled scrape with change-detection (Phase 3)
CDC guidance pages	Disease prevention and treatment guidance	Same as WHO	Same as WHO
ICMR / NHS regional guidelines	Region-specific clinical recommendations	Deferred to Phase 2 when multilingual/regional support is added	N/A in Phase 1

Dataset Note

PubHealth, SciFact, CoAID, and MedNLI are all real, published academic benchmarks with existing baselines in the literature — using them lets this project report results that are directly comparable to prior published work, which the four papers in the earlier research-gap review largely failed to do (Paper 2 used no dataset at all; Paper 4 relied partly on synthetic data).

6. Models — Detailed Plan

The reference document's model choices are reasonable starting points. This section keeps them where justified, adds licensing/sizing notes for MVP feasibility, and defines the new Faithfulness Verification model that the reference design lacks.

6.1 Claim Extraction

Option	Type	License	Recommendation
Qwen 3 (8B/14B instruct)	Open-weight LLM	Apache 2.0	Recommended for Phase 1 — strong instruction-following at a self-hostable size
Llama 3.1 (8B instruct)	Open-weight LLM	Llama 3.1 Community License	Strong alternative; widely supported tooling
Gemma 3 (9B/12B)	Open-weight LLM	Gemma license	Viable alternative; smaller footprint options available

6.2 Disease / Domain Classification

Model	Base	Fine-Tuning Data	Notes
BioBERT (recommended)	BERT, pretrained on PubMed abstracts + PMC full text	PubHealth + HealthFact, filtered/labeled by disease category	Best general-purpose biomedical classifier for Phase 1's 3-category scope

Model	Base	Fine-Tuning Data	Notes
PubMedBERT (alternative)	BERT, pretrained from scratch on PubMed text	Same as above	Marginal accuracy gains over BioBERT on some biomedical tasks; similar compute cost
ClinicalBERT (alternative)	BERT, pretrained on clinical notes (MIMIC-III)	MedNLI + curated set	Better suited if the project later expands into clinical-note-style input rather than social-media claims

6.3 Embedding Model (for Retrieval)

- Recommended: BAAI/bge-large-en-v1.5 — strong general-purpose retrieval embedding, open license, good performance on scientific/biomedical text in published benchmarks.
- Lightweight alternative for constrained environments: all-MiniLM-L6-v2 (also used as the sentence embedding choice in the earlier-reviewed Paper 4, giving a direct performance reference point).
- Phase 2 consideration: a biomedical-domain-tuned embedding model (e.g., a PubMedBERT-based sentence embedding) if generic BGE embeddings show weaker recall on niche disease terminology during evaluation.

6.4 Explanation-Generation LLM

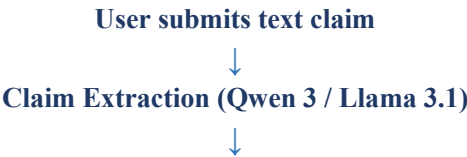
Same candidate set as claim extraction (Qwen 3 / Llama 3.1 / Gemma 3), self-hosted to avoid the per-call cost and vendor lock-in risk identified as a practical gap in the earlier review's analysis of Papers 2 and 4. A single 8B–14B model can serve both the claim-extraction and explanation-generation roles in Phase 1 to reduce infrastructure complexity; splitting into separate specialized models is a Phase 3 optimization, not a Phase 1 requirement.

6.5 New Addition — Faithfulness Verification Model (not present in the reference design)

This is the component that directly closes the hallucination gap identified across the earlier-reviewed literature. It runs after the LLM generates its explanation and before the explanation reaches the dashboard.

Approach	Model	Training/Evaluation Data	Output
Natural Language Inference (NLI) entailment check (recommended for Phase 1)	A MedNLI-fine-tuned entailment classifier (e.g., a BioBERT or DeBERTa-based NLI head)	MedNLI + SciFact rationale-sentence pairs	For each explanation sentence, an entailment/contradiction/neutral label against the retrieved evidence; sentences labeled “neutral” or “contradiction” relative to the cited evidence are flagged before display
LLM-as-judge cross-check (Phase 2 enhancement)	A second, independently-prompted LLM call asking “is this explanation supported by this evidence?”	Same evaluation sets, used to validate agreement with the NLI classifier	A secondary faithfulness score used to catch cases the NLI model misses; run only on flagged or high-stakes verdicts to control cost

7. System Architecture (Phase 1 MVP)





7.1 Why the Faithfulness Verification Step Is Placed Where It Is

Placing this check after explanation generation but before display means every explanation the user ever sees has already been screened for unsupported statements — rather than being an optional audit run after the fact. If a sentence fails the entailment check, the system either regenerates that sentence with a stricter grounding prompt or displays it with an explicit “unverified statement” marker, so the failure mode is visible rather than silent.

8. Database Design

8.1 Relational Store (PostgreSQL)

Table	Purpose	Key Fields
users	Account management (Phase 2+)	id, email, role, created_at
claims	Every submitted claim and its metadata	id, user_id, raw_text, disease_category, submitted_at
verifications	One row per completed verification	id, claim_id, verdict, credibility_score, consensus_summary, faithfulness_status, model_version
evidence_citations	Evidence items used per verification	id, verification_id, source, evidence_type, reliability_score, url_or_doi
feedback	User feedback on verdict quality (for active-learning loop, Phase 2)	id, verification_id, user_rating, comment
disease_categories	Reference table of supported categories and their model/config versions	id, name, active_phase, model_version

8.2 Vector Store

- Phase 1: FAISS — flat or IVF index, rebuilt on each scheduled knowledge-base update; simplest to operate for a single-host MVP.

- Fields stored alongside each vector: `document_chunk_text`, `source`, `publication_date`, `disease_category`, `evidence_type`, `reliability_score` (matching the reference document's design, retained as-is).
- Phase 3 migration trigger: move to Qdrant or Weaviate once metadata-filtered queries (e.g., “only RCTs published after 2020 for diabetes”) or multi-tenant access control become necessary — FAISS alone does not support rich metadata filtering efficiently at scale.

9. Evaluation Plan (absent from the reference document)

The reference document names datasets but never states how they validate the system. This section defines that missing evaluation protocol, directly addressing the experimental gaps identified in the earlier research review (task-mismatched metrics in Paper 2; an unexplained, possibly mismatched baseline table in Paper 4).

Component	Metric(s)	Dataset / Split	Target for Phase 1 Sign-Off
Disease classification	Accuracy, macro-F1 (to catch minority-class weakness, the specific failure seen in Paper 1 of the earlier review)	PubHealth + HealthFact, 70/15/15 train/val/test split	Macro-F1 ≥ 0.85 on held-out test set
Verdict classification (Supported/Contradicted/Insufficient)	Accuracy, precision/recall/F1 per class, calibration (Expected Calibration Error)	PubHealth test split (held out, never used in fine-tuning)	Report actual numbers; no target inflated by synthetic data, unlike Paper 4's evaluation practice
Evidence ranking quality	nDCG@k against SciFact's sentence-level supporting/refuting labels	SciFact test split	nDCG@10 ≥ 0.75
Faithfulness Verification	Precision/recall of the NLI entailment check against human-labeled hallucination examples	MedNLI test split + a manually annotated set of 200 real system outputs	Recall ≥ 0.90 on flagging unsupported statements (favor catching false explanations over minimizing false alarms)
End-to-end system	Human expert review (blind, spot-check sample) comparing system verdict to expert judgment	100–200 claims across the 3 Phase 1 disease categories, reviewed by qualified medical/nutrition consultants (as Paper 1 did in the earlier corpus)	$\geq 90\%$ agreement with expert majority verdict before Phase 1 sign-off

9.1 Baselines

- Baseline A: BioBERT-only classifier with no retrieval or explanation (reproduces the “existing system” described in the reference document's Section 6, i.e., BERT $\rightarrow$ True/False).
- Baseline B: Retrieval + LLM explanation with no Faithfulness Verification step (isolates the specific contribution of the new module added in this plan).
- Full system vs. both baselines, reported on the same held-out test sets — this ablation is the evidence needed to credibly claim the Faithfulness Verification layer is worth its added complexity.

10. Technology Stack

Layer	Reference Document Choice	This Plan (Phase 1)	Notes
Frontend	React + Tailwind CSS	Kept as-is	No change needed for MVP
Backend	Spring Boot	Kept as-is, or FastAPI-only if the team prefers a single-language stack	Spring Boot is fine if Java expertise is available; otherwise consolidating

Layer	Reference Document Choice	This Plan (Phase 1)	Notes
			to Python/FastAPI reduces context-switching for a small team
AI Service	FastAPI	Kept as-is	
Relational DB	PostgreSQL	Kept as-is	
Vector DB	FAISS	Kept as-is for Phase 1	Migration path to Qdrant/Weaviate defined in Section 8.2 for Phase 3
RAG orchestration	LangChain	Kept as-is	Add explicit logging of every retrieval/tool call, addressing the earlier review's finding that Paper 2's LangChain-agent claims were never actually evaluated with logged results
OCR	Tesseract	Deferred to Phase 2	No change to reference choice when implemented
Speech-to-Text	Whisper	Deferred to Phase 2	No change to reference choice when implemented
Deployment	Docker + Kubernetes	Docker Compose only for Phase 1–2	Kubernetes reintroduced in Phase 3 only if load testing justifies it

11. Security & Privacy Plan (absent from the reference document)

- Vector-store integrity monitoring: before any automatic knowledge-update job (Section 4, Phase 3) is allowed to write new embeddings unsupervised, new entries are checked for embedding-space anomalies relative to the existing verified-fact cluster, addressing the vector-database poisoning risk identified in the earlier research review.
- User data minimization: submitted claims are stored without directly identifying information by default; account linkage (Phase 2) is opt-in and covered by an explicit data-retention policy.
- API key and rate-limit management: PubMed API and any hosted-LLM fallback calls go through a centralized gateway with rate limiting and logging, preventing both abuse and unexpected cost overruns — an operational gap left unaddressed in the reference document.
- Adversarial input handling: claim-extraction and classification inputs are sanitized against prompt-injection patterns before being passed to the LLM layer; flagged inputs are logged for review rather than silently processed.
- Model/version provenance: every stored verification result records the exact model versions used (Section 8.1's `model_version` field), so past verdicts remain explainable even after models are upgraded.

12. Risk Register

Risk	Likelihood	Impact	Mitigation
Evidence-ranking scores remain unvalidated heuristics	Medium	High — undermines the system's core credibility claim	Mandatory calibration study against SciFact (Section 9) before Phase 1 sign-off
Live PubMed API latency/rate limits slow down verification	Medium	Medium	Cache recent queries; fall back to static indexed corpus when live API is slow or unavailable
Self-hosted LLM (8B–14B) explanation quality lower than GPT-4	Medium	Medium	Evaluate explanation quality against a small GPT-4 comparison set during Phase 1 testing; upgrade model size only if evaluation shows a material quality gap

Risk	Likelihood	Impact	Mitigation
Faithfulness Verification module produces too many false-positive flags, frustrating users	Medium	Low-Medium	Tune the NLI classifier's decision threshold using the held-out annotated set (Section 9) before launch
Scope creep back toward the full 12-module reference design before Phase 1 is validated	High	High — repeats the over-promising pattern seen in the earlier-reviewed literature	Phase gate: Phase 2 work does not start until Phase 1 evaluation targets (Section 9) are met and signed off

13. Summary

This plan keeps everything genuinely strong in the MedVerify AI reference design — reliability-weighted evidence ranking, hybrid static+live retrieval, and consensus analysis — while fixing its two biggest weaknesses: an unbounded, unphased scope, and the absence of any check on whether its own LLM-generated explanations are actually true. Both fixes are grounded directly in gaps identified across the earlier four-paper research review, giving this implementation a defensible answer to “why is this different from what already exists”: it is the first design in this line of work to combine evidence grounding, reliability weighting, and faithfulness verification in one pipeline, shipped as a scoped, evaluable MVP rather than an all-at-once feature list.

Problems for the Implementation

My diagnosis, in one sentence

Every one of these "complications" is actually an unsolved measurement problem, not a coding problem. The fix for all of them is the same discipline: separate "does this look related" (retrieval) from "should I trust it" (reliability) from "what does the AI think" (generation) — and never let one silently stand in for another. Once you internalize that separation, most of the 15 issues in that document resolve into the same handful of design decisions.

How I'd actually solve the five hardest ones

1. Evidence ranking (relevance ≠ reliability) — the one everything else depends on

Don't try to invent one clever formula. Build it in three layers, and don't skip the middle one:

- **Layer A — Retrieval (cheap, fast):** FAISS/embedding similarity finds *candidates*. Its only job is recall, not trust. Never let this score touch the final credibility number.
- **Layer B — Reliability scoring (transparent, rule-based, versioned):** score each candidate independently of the claim, using structured signals you can actually defend: source-type tier (meta-analysis > RCT > cohort > case report > preprint > blog), peer-review status, recency (with a decay half-life you tune per disease — fast-moving areas like infectious disease need shorter half-lives than stable ones like diabetes pathophysiology), and — if available — citation count via Semantic Scholar's API. Write this as one documented function, version it, and never hand-wave it.
- **Layer C — Calibration (the part almost everyone skips):** once you have a score, you have to prove it means something. Run it against SciFact's human-labeled supporting/refuting sentences and check: do higher-scored documents actually align with correct verdicts more often? Use isotonic regression or Platt scaling to turn your raw score into something calibrated, not just "a number that goes up."

If you only do Layer A and skip B and C, you get exactly the reviewer question ChatGPT flagged: “*why 87?*” — and you won't have an answer.

2. Consensus — weight it, don't count it

Replace "7 supporting, 3 contradicting" with a reliability-weighted score:

$$\text{consensus} = \Sigma(\text{reliability}_i \times \text{stance}_i) / \Sigma(\text{reliability}_i)$$

where stance is +1/−1/0. One meta-analysis at reliability 0.95 should outrank seven case reports at reliability 0.2. Critically — **show both numbers in the UI**, the raw distribution and the weighted consensus, labeled separately. That transparency is what makes it defensible to a panel instead of looking like you buried the disagreement.

Add one more thing ChatGPT didn't mention: if the evidence is genuinely heterogeneous (studies disagree a lot, not just in count but in effect direction/size), your system should be willing to say "**conflicting evidence — no clear consensus**" rather than forcing a verdict. That's what real systematic reviews do (this is the same instinct behind Cochrane's "insufficient evidence" conclusions), and it's more honest than a false-confidence 62%.

3. Faithfulness — the causation/association trap is real, and it's tractable

This is the sharpest point in that whole document, and it's correct: a generic NLI model won't reliably catch "associated with" → "causes." Here's the concrete fix:

- Add a **hedge/certainty detector** as a separate signal alongside NLI — this is a well-studied biomedical NLP subtask (look up the **BioScope corpus**, built exactly for speculation/negation/certainty detection in biomedical text). Extract the certainty level of the evidence sentence ("may," "associated with," "suggests" = weak; "causes," "confirmed," "proven" = strong) and the certainty level of the LLM's explanation sentence. Flag any case where the explanation's certainty exceeds the evidence's certainty.
- Add a **generation-time constraint** too, not just post-hoc checking — explicitly prompt the LLM: *"preserve the evidence's original certainty language; do not upgrade 'associated with' to 'causes.'"* Cheap defense-in-depth before the verification layer even runs.
- Be honest with yourself: NLI + hedge detection is a **faithfulness signal**, not a truth oracle. Keep a human-expert spot-check on a sample of verdicts even after this ships — full automation of medical trust claims is genuinely risky, and a defensible system says so rather than pretending to be 100% automated.

4. Evaluation / data leakage — clarify what "leakage" actually means here

One nuance ChatGPT's document blurs: it's *fine and expected* for the correct evidence document to be sitting in your FAISS index during a retrieval evaluation — that's what nDCG is measuring, can the system find it. The real leakage risk is narrower and more specific:

- Never fine-tune the disease classifier on claims that are in your held-out test split.
- Never use test-split claims as few-shot examples in your LLM prompts.
- Hash and lock your test sets before Phase 1 sign-off, and keep a simple log (even a spreadsheet) of dataset versions used per experiment — this is what lets you defend your numbers six months later.

5. Scope — agreed completely, nothing to add. The phase-gate discipline from the implementation plan stands.

What's my actual next move

Here's the part I'd change about how you approach the next two weeks: **don't build the app around these five problems — prove each one works as a standalone experiment first.**

1. **Week 1, before touching FastAPI or Spring Boot:** a single notebook. Take 30–50 SciFact claims, compute your reliability score, and check whether it actually correlates with SciFact's human labels. If it doesn't, you've saved yourself from building a product around a broken formula.

2. **Week 2, same notebook style:** run any LLM on 20–30 real claims, manually label which sentences are hallucinated vs. faithful, and test whether NLI + hedge-detection actually catches them. Measure precision/recall on this tiny set.
3. **Only once both of those show real signal** do you start wiring up the pipeline, backend, and UI from the implementation plan.

This is standard research-engineering discipline: de-risk the science in a notebook before you invest in software around it. It also gives you your first real experimental results to put in a paper.

Dataset integration & training at scale — the risks you're actually asking about

This is the part worth being most careful about, because the instinct to "train on lots of data" is where I'd slow you down.

You don't need a huge dataset for the credibility score — you need a well-audited small one. The credibility score is a *calibrated, composed function*, not something you train end-to-end on millions of examples. Paper 1 from your earlier research review made this exact point: 10,000 medical-expert-labeled entries beat larger, noisier data. Chasing volume here is a trap.

Specific risks as you scale up:

Risk	What actually happens	What to do
Label-quality mismatch	PubHealth's labels come from journalist fact-checkers, not medical experts — different standard than Paper 1's 3-expert majority vote	Audit a sample of PubHealth labels against a medical consultant before trusting them at scale; don't assume "published dataset" = "medically correct label"
Distribution shift	Models trained only on curated fact-check datasets (formal, well-written claims) underperform on real sample of real social posts before production — exactly what Twitter/Reddit-style text (typos, slang, sarcasm)	Fine-tune further on a small (even a few hundred), expert-labeled sample of real social posts before production — exactly what
Class imbalance at scale	More data usually means <i>more</i> imbalance (most real claims skew "true" or "insufficient evidence"), not less	Track per-class recall, not aggregate accuracy — this is precisely where Paper 1's LSTM quietly failed (34% recall on the False class despite 90% overall accuracy)
Compute misallocation	The temptation is to fine-tune the LLM itself on more data	Don't. Keep the LLM prompt-only. Spend your actual training budget on the small, cheap classifiers (BioBERT-scale, NLI-scale) — that's where fine-tuning genuinely pays off and where GPU cost is trivial
Feedback-loop contamination	Auto-retraining on raw user feedback (Phase 2's active-learning loop) risks the model reinforcing its own mistakes	Route all feedback through expert review before it ever touches a retraining set — never retrain directly on unverified user signals
Scraping at scale hits legal/ToS limits fast	"Lots of data" from Twitter/Reddit at Lean research-project scale risks limiting or API bans before you get useful volume	on existing academic datasets (PubHealth/SciFact/CoAID/MedNLI) as your backbone; treat social-media scraping as a small, targeted supplement, not your primary data source